Deception-Based Security Mechanism for Detecting Unauthorised Access

1. Introduction

Modern cybersecurity systems rely heavily on preventive controls such as firewalls, authentication mechanisms, and intrusion detection systems. However, these mechanisms are often insufficient when attackers bypass initial defences or operate stealthily within the system. In such cases, identifying malicious intent becomes difficult using traditional approaches.

Deception-based security introduces a different approach by placing deliberately crafted fake resources within a system. These resources appear legitimate but are not intended for normal use. Any interaction with such components is considered suspicious and can be used to identify potential attackers.

This project presents the design and implementation of a deception-based security system that includes:

- **A spoofed login interface**
- **A dummy internal API**
- **A monitored honeypot file**

The system continuously monitors these components and generates alerts whenever unauthorised interaction occurs.

2. Objective

The primary objectives of this project are:

- To design deceptive components that mimic real system elements
- To monitor access attempts to these components
- To identify suspicious or malicious behaviour based on interaction
- To generate logs and alerts for further analysis

3. System Overview

The system is implemented using Python and Flask, and it consists of three main deception elements:

1. **Fake Login Interface**
   A web-based login panel designed to capture unauthorised credential attempts.
2. **Fake Internal API**
   A non-functional API endpoint that simulates a sensitive backend service.
3. **Honeypot File**
   A file containing fake credentials that is monitored for unauthorized access.

All interactions with these components are recorded in a centralized log file (alerts.log) and displayed in real time on the terminal.

4. System Architecture

4.1 Architecture Description

The system follows a simple monitoring architecture where deceptive components act as traps and a logging module records all suspicious activity.

4.2 Architecture Diagram

```
+----------------------+
|   User / Attacker    |
+----------+-----------+
           |
           v
+-----------------------------+
| Deception Components        |
|-----------------------------|
| 1. Fake Login (Port 5000)   |
| 2. Fake API (Port 7000)     |
| 3. Honeypot File            |
+--------------+---------------+
               |
               v
+-----------------------------+
| Detection & Monitoring Layer |
|-----------------------------|
| Flask Applications          |
| File Monitoring Script      |
```

```
+--------------+---------------+
             |
             v
+-----------------------------+
| Logging System              |
|-----------------------------|
| alerts.log                  |
| Real-time Terminal Alerts   |
+-----------------------------+
```

---

5. Implementation Details

5.1 Fake Login Interface

A Flask application (fake_login.py) is used to simulate an admin login page.

- The login page accepts username and password input.
- No actual authentication is performed.
- Every login attempt is treated as suspicious.

When a user submits credentials:

- The system logs the attempt
- Displays an alert in the terminal

Sample Output:

ALERT: Fake login triggered!
Fake Login Attempt: admin

Figure 1: Fake Login Page Interface

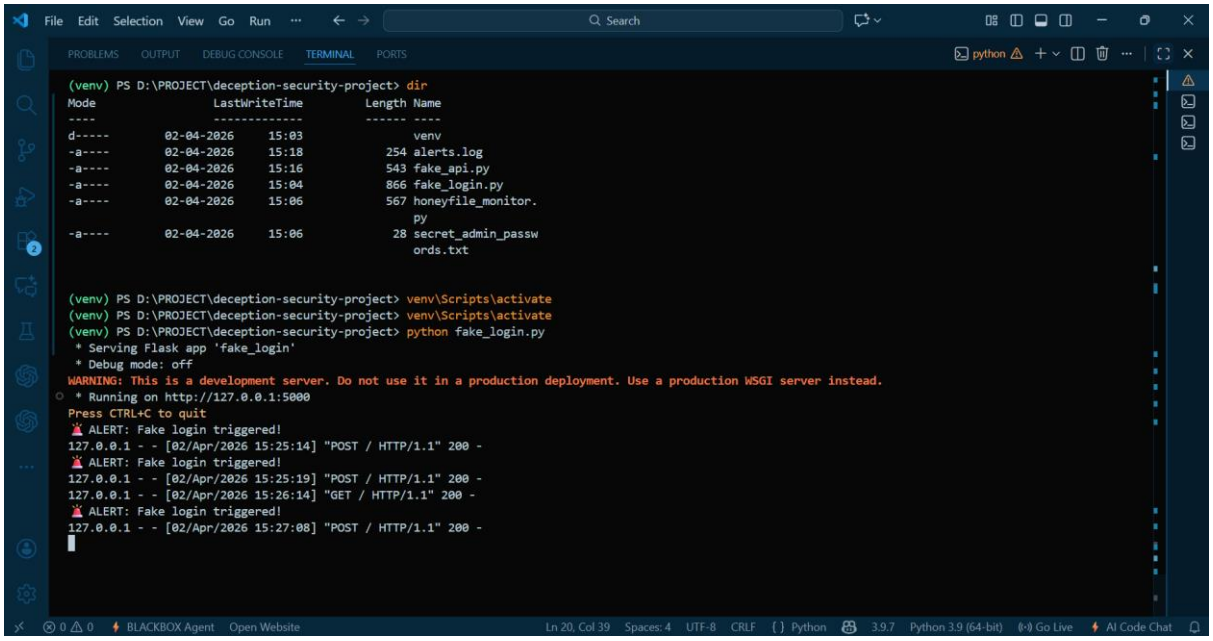

Figure 2: Fake Login Alert in Terminal

5.2 Fake Internal API

A second Flask application (fake_api.py) simulates an internal API endpoint.

- Endpoint: /internal-Api
- Runs on port 7000
- Any request is treated as unauthorised access

Response Returned:

{"message": "Unauthorised access detected"}

Terminal Output:

ALERT: Fake API accessed!
GET /internal-Api HTTP/1.1 403

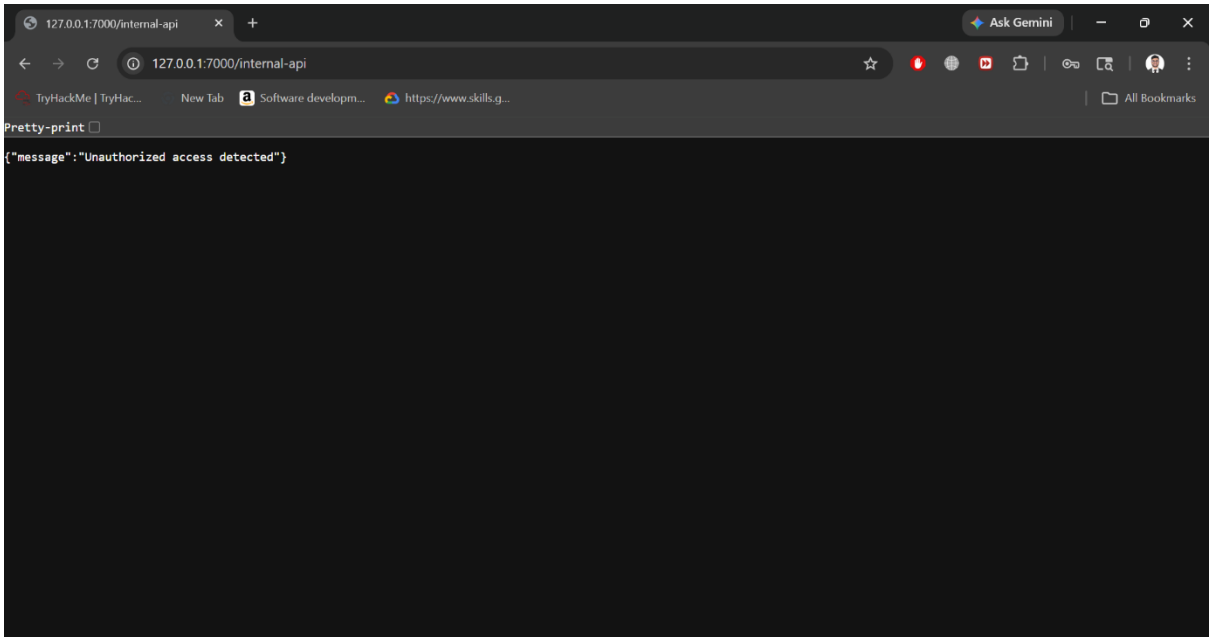

Figure 3: Fake API Response in Browser

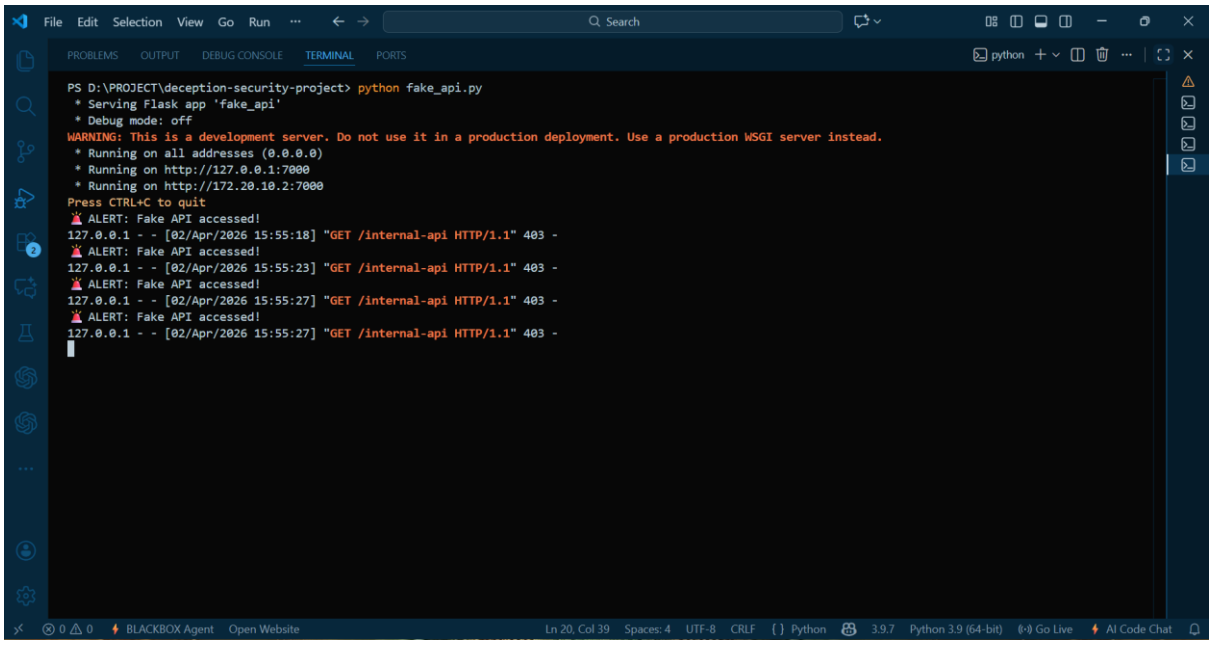

Figure 4: API Access Alert in Terminal

5.3 Honeypot File Monitoring

A file named:

secret_admin_passwords.txt

is placed in the system to simulate sensitive data.

Contents:

admin:password123
root: Toor

A monitoring script (honeypile_monitor.py) continuously checks for file access.

When accessed:

- The system logs the event
- Displays an alert

Sample Output:

ALERT: Honey pile accessed!

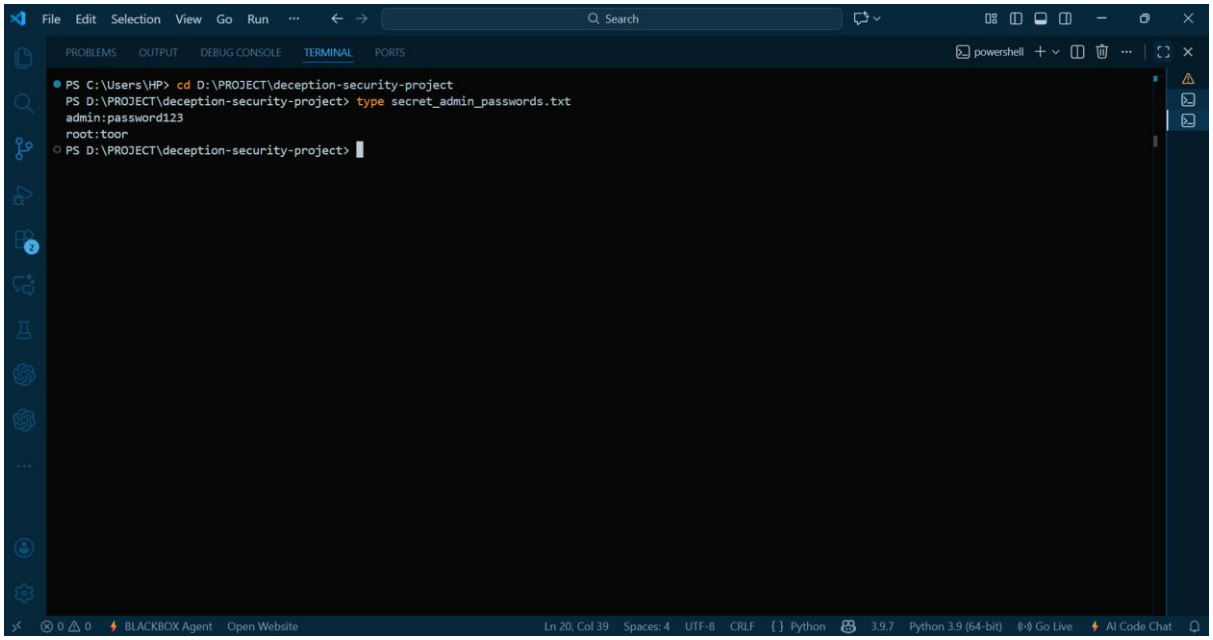

Figure 5: Honeypot File Content

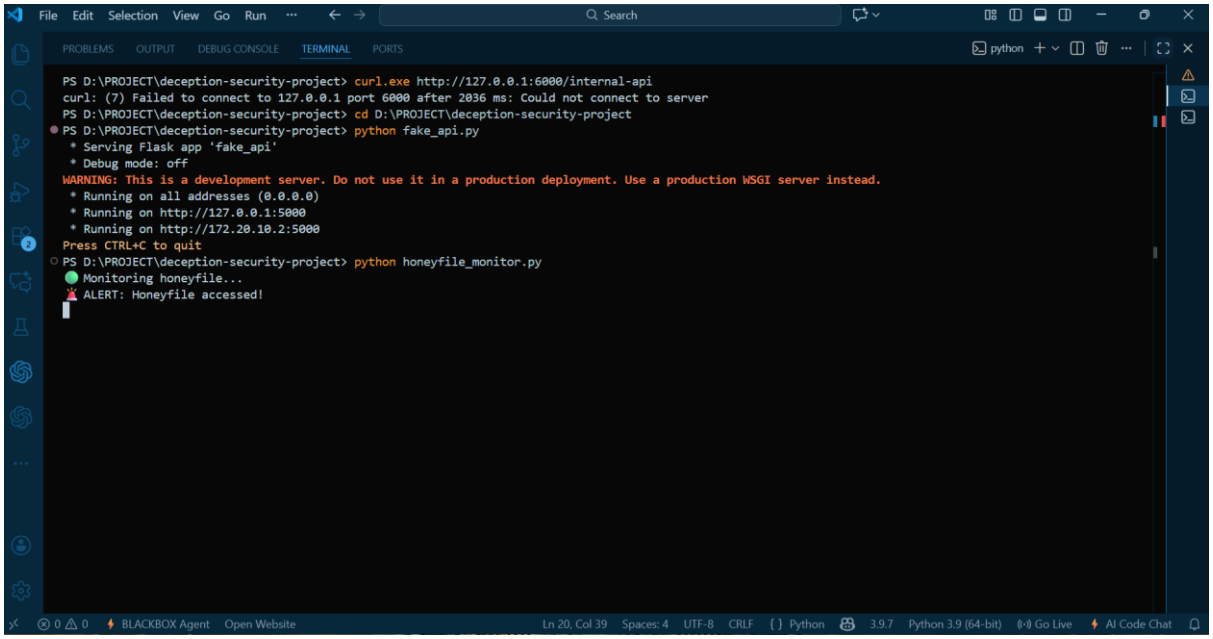

Figure 6: Honeypot Access Alert

5.4 Logging Mechanism

All suspicious activities are stored in:

alerts.log

Each log entry contains:

- Timestamp
- Source IP (if applicable)
- Event description

Sample Log Entries:

2026-04-02 15:05:07 | 127.0.0.1 | Fake Login Attempt
2026-04-02 15:07:52 | Honey pile accessed
2026-04-02 15:10:28 | Accessed /internal-Api

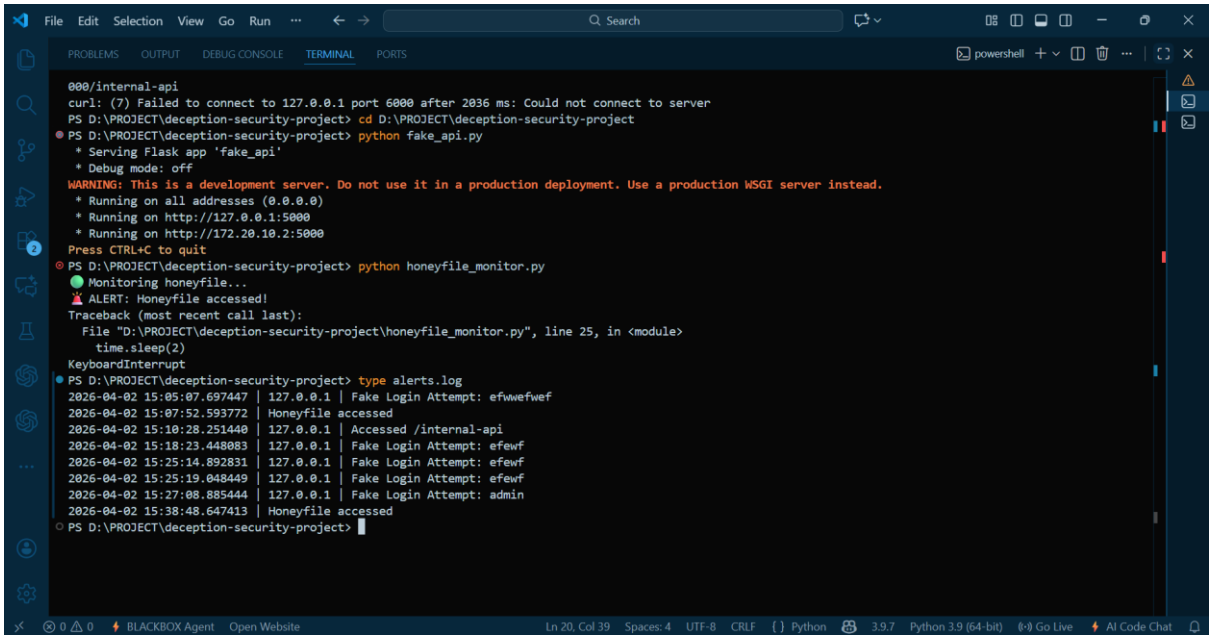

Figure 7: alerts.log Output

5.5 Important Code Snippets

Fake Login Detection

```
@app.route("/", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form["username"]
        log_event(f"Fake Login Attempt: {username}")
        print("ALERT: Fake login triggered!")
```

Fake API Detection

```
@app.route("/internal-api")
def fake_api():
    log_event("Accessed /internal-api")
    print("ALERT: Fake API accessed!")
    return {"message": "Unauthorized access detected"}, 403
```

Honeypot Monitoring

```
import os, time

file_path = "secret_admin_passwords.txt"
last_access = os.path.getatime(file_path)

while True:
    current_access = os.path.getatime(file_path)
    if current_access != last_access:
        print("ALERT: Honeypot accessed!")
```

Logging Function

```
from datetime import datetime

def log_event(event):
    with open("alerts.log", "a") as f:
        f.write(f"{datetime.now()} | {event}\n")
```

---

6. Detection Strategy

6.1 Detection Logic

The system detects unauthorised activity based on interaction with deceptive components:

- The fake login page is not part of the normal workflow, so any login attempt is flagged as suspicious.
- The internal API endpoint is hidden and not publicly documented, so any request is considered unauthorised.
- The honeypot file is intentionally placed as bait; accessing it indicates potential malicious intent.
- The system identifies suspicious behaviour based on predefined conditions:

Component	Trigger Condition	Action
Fake Login	Any login attempt	Log + Alert
Fake API	Any API request	Log + Alert
Honeypot File	File access	Log + Alert

This ensures that any interaction with deceptive elements is immediately flagged.

---

7. Testing and Results

The system was tested using multiple simulated attack scenarios:

7.1 Login Attempt Testing

- Entered multiple usernames and passwords
- System successfully logged all attempts

7.2 API Access Testing

- Accessed /internal-Api using browser and curl
- System detected and logged all requests

7.3 File Access Testing

- Opened the honeypot file via the terminal
- Alert generated instantly

7.4 Observations

- All malicious interactions were detected successfully
- Alerts were generated in real time
- Logs were properly structured and readable

7.5 Sample Session Logs

2026-04-02 15:25:14 | 127.0.0.1 | Fake Login Attempt: admin

2026-04-02 15:27:08 | 127.0.0.1 | Fake Login Attempt: testuser

2026-04-02 15:55:18 | 127.0.0.1 | Accessed /internal-api

2026-04-02 15:07:52 | Honeypot accessed

---

8. Security Analysis

8.1 Strengths

- Detects attacker behaviour instead of relying on signatures
- Simple and effective implementation
- Real-time monitoring capability
- Useful for early-stage intrusion detection

8.2 Limitations

- Limited to local environment
- No automated blocking mechanism
- Does not integrate with enterprise security tools
- File monitoring is basic and can be improved

---

9. Future Enhancements

The system can be extended with:

- Integration with EDR tools (e.g., Microsoft Defender, CrowdStrike)
- Automated response (blocking IP addresses)
- Web dashboard for monitoring
- Email alert system
- Deployment on cloud infrastructure

---

10. Conclusion

This project demonstrates how deception techniques can be used to detect unauthorized access attempts effectively. By deploying fake components such as a login interface, internal API, and honeypot file, the system is able to identify suspicious behaviour in real time.

The results show that deception-based security can complement traditional defences by focusing on attacker interaction rather than known attack patterns. This approach is especially useful in identifying insider threats and unknown attack methods.

---

11. References

---

1. OWASP Foundation – Deception Technologies
2. Flask Documentation – https://flask.palletsprojects.com/
3. Python Official Documentation – https://docs.python.org/3/
4. Spitzner, Lance. *Honeypots: Tracking Hackers*
5. NIST Cybersecurity Guidelines

6. 6. NIST SP 800-92 – Guide to Computer Security Log Management